

VI

PREUVES FORMELLES



INTRODUCTION

- Les méthodes formelles prennent tout leur intérêt lorsque les preuves elles-mêmes sont garanties correctes formellement.
- On peut distinguer deux grandes catégories d'outils permettant la preuve de propriétés sur des modèles formels :
 - la preuve automatique de théorème, qui consiste à laisser l'ordinateur prouver les propriétés automatiquement, étant donné une description du système, un ensemble d'axiomes et un ensemble de règles d'inférences.
 - le model checking, qui consiste à vérifier des propriétés par une énumération exhaustive et astucieuse (selon les algorithmes) des états accessibles.



PREUVE AUTOMATIQUE DE THÉORÈME

- La preuve automatique de théorème, qui consiste à laisser l'ordinateur prouver les propriétés automatiquement, étant donné une description du système, un ensemble d'axiomes et un ensemble de règles d'inférences.
- Cependant la recherche de preuve est connue pour être un problème non décidable en général (en logique classique), c'est-à-dire que l'on sait qu'il n'existe (et n'existera jamais) aucun algorithme permettant de décider en temps fini, pour toute propriété, si cette propriété est vraie ou fausse.



PREUVE AUTOMATIQUE DE THÉORÈME

- Il existe des cas où le problème est décidable (fragment gardé de la logique du premier ordre par exemple) et où des algorithmes peuvent donc être appliqués.
- Cependant même dans ces cas, le temps et les ressources nécessaires pour que les propriétés soient vérifiées peut dépasser des temps acceptables ou les ressources dont dispose l'ordinateur.



PREUVE AUTOMATIQUE DE THÉORÈME

- . Il existe des outils interactifs qui permettent à l'utilisateur de guider la preuve.
- . La preuve de théorème, par rapport au model-checking, a l'avantage d'être indépendante de la taille de l'espace des états, et peut donc s'appliquer sur des modèles avec un très grand nombre d'états, ou même sur des modèles dont le nombre d'états n'est pas déterminé (modèles génériques) ;



QU'EST CE QU'UNE OBLIGATION DE PREUVE ?

- Une obligation de preuve (PO) est constituée d'un but B et d'un ensemble d'hypothèses H .
- Démontrer une PO consiste à démontrer ce but B , en supposant que toutes les hypothèses de H sont vérifiées.
- Les hypothèses H sont dites hypothèses contextuelles.
- Si le but courant est de la forme $P \Rightarrow Q$, par application du principe de déduction, P va devenir une hypothèse et le nouveau but devient Q . P est alors appelé hypothèse dérivée.



QU'EST CE QU'UNE OBLIGATION DE PREUVE ?

- Les hypothèses courantes sont constituées des hypothèses contextuelles et des hypothèses dérivées.
- La pile des hypothèses contient l'ensemble des hypothèses contextuelles et des hypothèses dérivées, dans l'ordre dans lequel elles sont apparues.

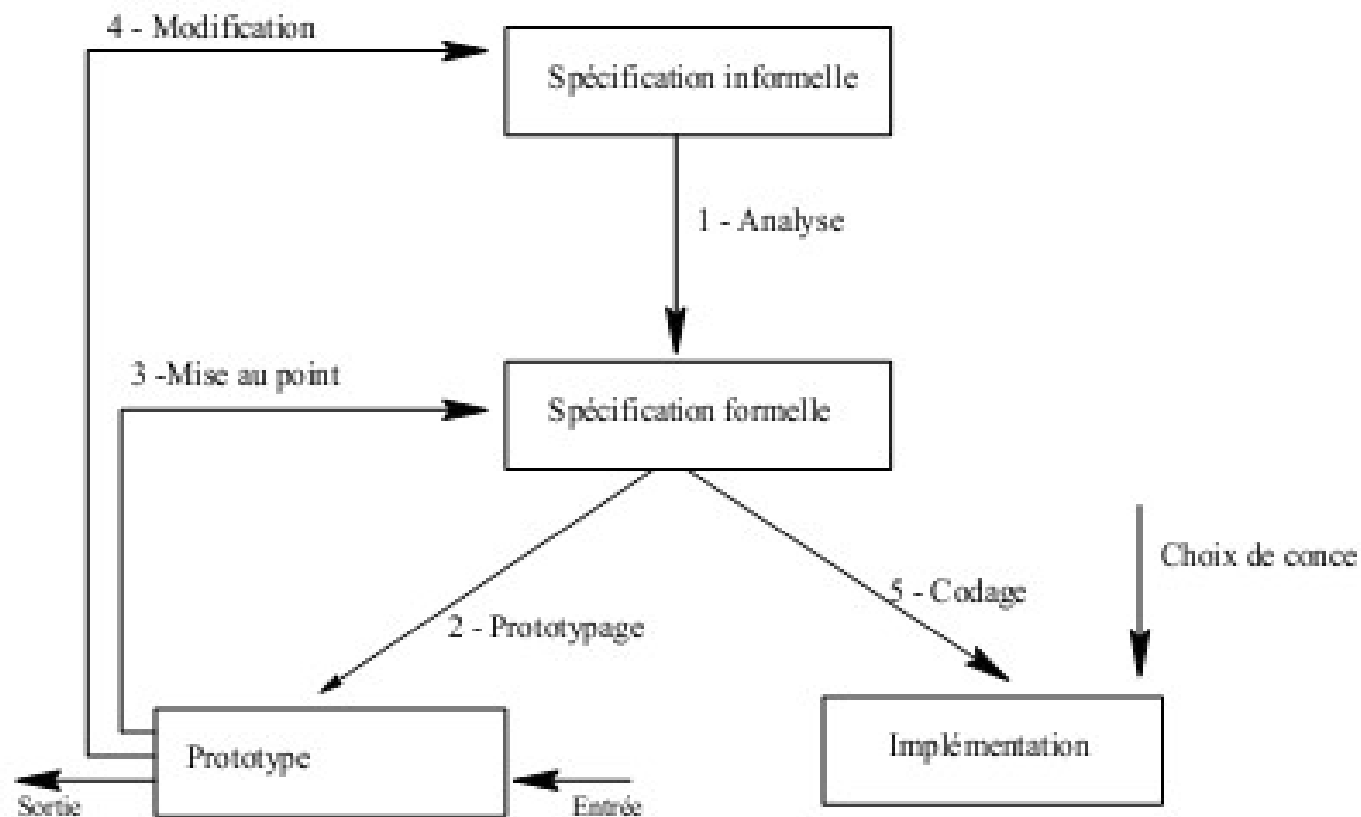


QU'EST CE QU'UNE OBLIGATION DE PREUVE ?

- les obligations de preuve sont générées automatiquement par des outils
- On peut avoir plusieurs milliers d'obligations de preuve
- On peut avoir plusieurs centaines d'hypothèses par obligation de preuve (voire plus)



QU'EST CE QU'UNE OBLIGATION DE PREUVE ?



QU'EST CE QU'UNE OBLIGATION DE PREUVE ?

Quand et pourquoi sont-elles générées ?

- pour garantir la sûreté du programme, dès qu'une opération peut entraîner une erreur à l'exécution
 - divisions par zéro
 - déréférencement des pointeurs
 - overflow des opérations arithmétiques
- pour vérifier les spécifications utilisateurs
 - post-conditions
 - assertions
 - invariants et variants de boucles
 - ...



QU'EST CE QU'UNE RÈGLE ?

- Une règle est une formule qui se présente sous la forme $A \Rightarrow B$.
 - A est appelé antécédent de la règle.
 - B est appelé conséquent de la règle.
 - A et B peuvent être des conjonctions de prédicats.
 - A peut être omis. Dans ce cas, la règle est dite atomique.
- Une règle peut être :
 - inductive (backward) : Si le but courant est B, alors pour prouver B, il suffit de prouver A. A est sensé être plus simple que B ou, du moins, plus facilement prouvable que B.
 - déductive (forward) : Si les hypothèses A apparaissent dans la pile des hypothèses, alors les hypothèses B sont générées et montées dans la pile, si elles n'existent pas déjà.



QU'EST CE QU'UNE RÈGLE ?

- Une règle peut être :
 - de réécriture : dans ce cas, B est de la forme $C == D$. Si A est vérifié alors C est réécrit en D. Ce type de règle ne s'applique que sur des sous-formules contenues dans le but courant, ou sur le but courant lui-même. **Exemple** : $(p < q) \Rightarrow \max(p, q) == q$
- Les règles, contrairement aux hypothèses et au but, contiennent des jokers.
 - Un joker est une variable, qui peut prendre n'importe quelle valeur (littéral, expression, ...).
 - Si on lui affecte une valeur, on dit alors qu'il est instancié.
 - Un joker est représenté par une lettre de l'alphabet : on ne peut donc pas avoir plus de 52 jokers à l'intérieur d'une même règle (majuscules et minuscules).



COMMENT EFFECTUER UNE PREUVE ?

- Prouver à la main (comme n'importe quel théorème de maths)
 - impossible en pratique
- Prouver à l'aide d'un assistant de preuves : outil pour assister un humain dans l'activité de preuves en le guidant, en résolvant les parties triviales ou calculatoires, et en vérifiant leur validité
 - Exemple : Coq, Isabelle, PVS, etc.
- Prouver avec des prouveurs automatiques : outil “presse-bouton” effectuant les preuves de manière automatique
 - Exemple : B, Simplify, Z3, etc.



MÉTHODE B



INTRODUCTION

- Méthode proposé en 1996 pour spécifier, concevoir et développer des systèmes informatiques séquentiels
- Une variante appelée Event B a été proposée en 1998 pour les systèmes distribués, concurrents
- B utilise son propre langage formel. Au niveau des spécifications, il s'agit d'un langage logique reposant sur une version simplifiée de la théorie des ensembles



INTRODUCTION

- Les modèles traité par la méthode sont caractérisés par
 - un état i.e. ensemble de variables
 - ensemble d'actions pouvant modifier l'état i.e. les variables
- Les principales préoccupations sont :
 - Quelles sont les propriétés satisfaites par ces modèles ?
 - Comment prouver ces propriétés ?
 - Comment obtenir du code à partir de ces modèles ?



ENSEMBLES DE BASE

- Le langage de B permet également de manipuler des ensembles.
 - $\emptyset$ $\{\}$ L'ensemble vide
 - N NAT L'ensemble des entiers naturels
 - Z INTEGER L'ensemble des entiers relatifs
 - INT INT L'ensemble des entiers relatifs compris entre MININT et MAXINT
 - $\{e\}$ $\{e\}$ L'ensemble réduit à un élément e
 - $\{e_1, \dots, e_n\}$ $\{e_1, \dots, e_n\}$ L'ensemble composé des éléments $e_1, \dots, e_n$
- Exemple : $\{1, 2, 3\}$ est un ensemble. $\{\emptyset\}$ est un ensemble composé d'un élément.
- **Compréhension**

Soit P une formule, on peut définir un ensemble par compréhension comme l'ensemble de tous les objets qui vérifient la formule P . On note cet ensemble $\{z \mid P\}$.

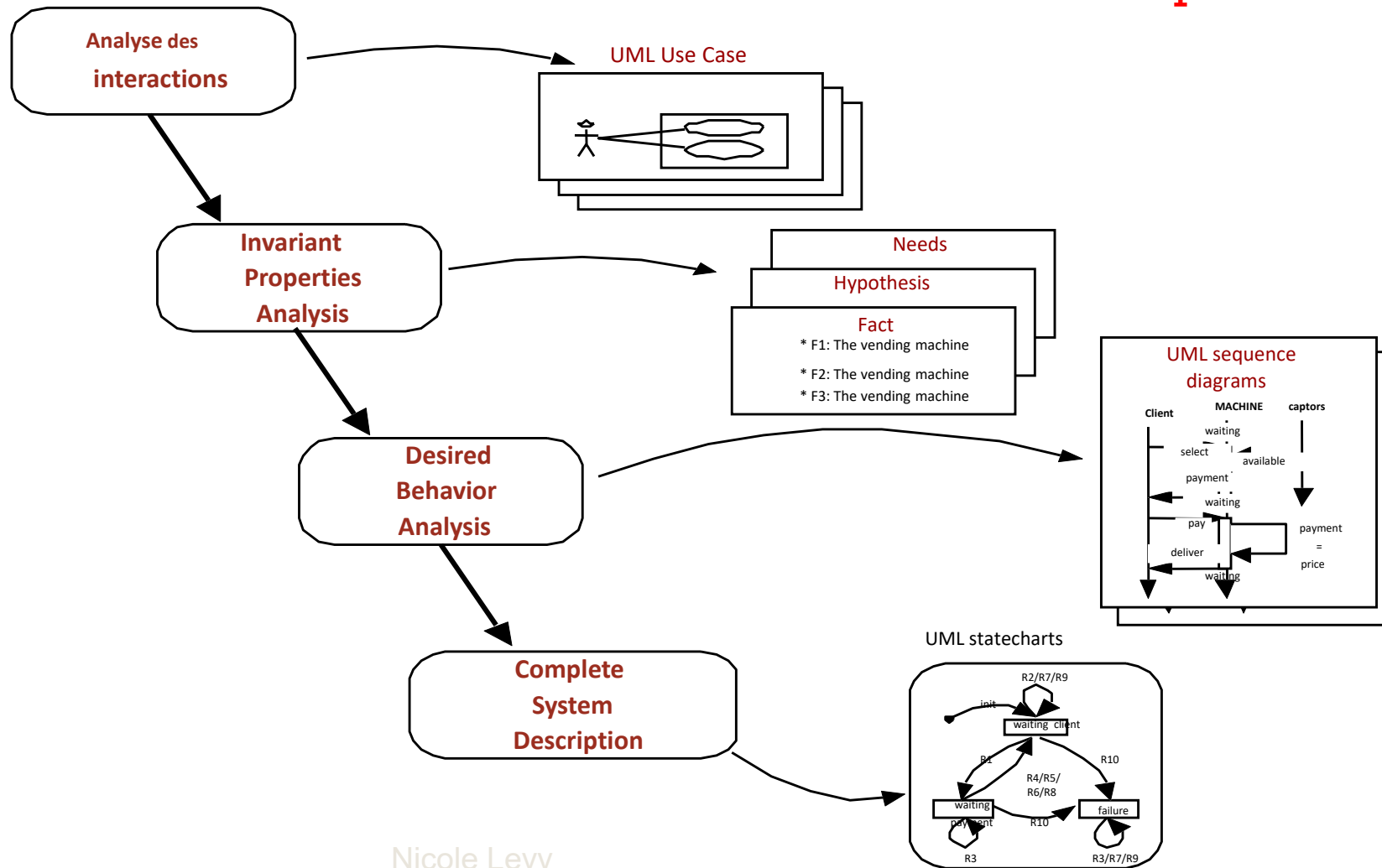


DÉVELOPPEMENT DE PROGRAMMES AVEC B

- Développement formel de programmes impératifs à partir de spécification
- Méthode
 - Raffinement de la spécification jusqu'au code.
 - Succession de modèles liés par une relation de raffinement
 - Le dernier modèle est écrit dans un langage proche du code
- Etapes
 - Analyse système
 - Structuration : Machines abstraites
 - Modélisation données
 - Modélisation opérations
 - Raffinements
 - Implantation



Étapes d'analyse



DÉVELOPPEMENT DE PROGRAMMES AVEC B : LA SPÉCIFICATION

La spécification définit la Machine Abstraite

- Décrire les variables d'état du système,
- Exprimer l'invariant i.e. propriétés satisfaites par les variables d'état à l'initialisation, avant et après toute opération manipulant les variables d'état,
- Spécifier enfin ces opérations



DÉVELOPPEMENT DE PROGRAMMES AVEC B : LE RAFFINEMENT

Le raffinement permet :

- Décrire (ou réécrire) les variables d'état du modèle raffiné
- Exprimer l'invariant i.e. propriétés satisfaites par les (nouvelles) variables d'état + l'invariant de collage à l'initialisation avant et après toute opération manipulant les variables d'état
- Réécrire (préciser) les opérations avec les (nouvelles) variables
- **NB**, certaines constructions ne sont pas permises au niveau de la machine et des raffinements

⇒ Besoin d'exprimer, ensembles, propriétés, opérations et transformations d'états, machines et raffinements.



LE LANGAGE DE MODÉLISATION

- Permet l'expression de variables, constantes, opérations, invariants, théorèmes, . . .

- Se base sur :

1. Théorie des ensembles

$\text{SIEGES} = 1..nb_max$

$\text{occupes} \subseteq \text{SIEGES}$

2. Fonctions, relations

$\text{etat} \in \text{SIEGES} \rightarrow \text{BOOL}$

etat^{-1} fonction réciproque.



LE LANGAGE DE MODÉLISATION

3. Logique des prédicats

`occupes = etat-1 [{TRUE}]`

`x ∈ 1..1280 ∧ y ∈ 1..1024`

4. Des opérations sont définies pour manipuler et construire des ensembles, relations, fonctions, et prédicats



LE LANGAGE DE MODÉLISATION

- **Substitutions généralisées** : permettent de décrire les changements d'états

```
ANY  
   $x', y'$   
WHERE  
   $x' \in 1..1280 \wedge y' \in 1..1024$   
THEN  
   $x := x' \parallel y := y'$   
END ;
```

```
PRE  
   $x + 100 \leq 1280 \wedge y + 100 \leq 1024$   
THEN  
   $x := x + 100 \parallel y := y + 100$   
END ;
```



LE LANGAGE DE MODÉLISATION

- **Substitutions généralisées** : permettent la modification d'une variable

Notation	Nom
<i>skip</i>	substitution identité
$x := e$	substitution devient égal simple
$x, y := e, f$	substitution devient égal multiple
$x := e \parallel y := f$	
$x : (P)$	substitution devient tel que



LE LANGAGE DE MODÉLISATION

- **Substitutions généralisées** permettent de structurer une opération

Syntaxe de la substitution	Nom de la substitution
BEGIN <i>S</i> END	bloc
PRE <i>P</i> THEN <i>S</i> END	pré-condition
ANY <i>I</i> WHERE <i>G</i> THEN <i>S</i> END	choix non borné
...	...



CALCUL DES SUBSTITUTIONS GÉNÉRALISÉES

- Obligation de preuve : théorème à prouver
- Triplet de Hoare $\{P\} S \{Q\}$
- Weakest Pre-condition (WP) de Dijkstra $P \equiv [S]Q$ (Plus faible pré-condition P qui établit Q après S)
- Calcul de plus faibles pré-conditions $\Leftrightarrow$ Principe d'extraction des obligations de preuve (OP)

$[S]Q$	WP
$[x := E]Q$	$Q(E \rightarrow x)$
Exemple $[x := y + 5](x > y)$	$y + 5 > y$

$[S]Q$	WP
$[\text{PRE } P \text{ THEN } S \text{ END}]Q$	$P \wedge [S]Q$
Exemple $[\text{PRE } x > 0 \text{ THEN } x := y + 5 \text{ END}](x > y)$	$(x > 0) \wedge (y + 5 > y)$



STRUCTURE GLOBALE D'UNE MACHINE ABSTRAITE

MACHINE *nom de la machine abstraite*

Partie statique

déclaration d'ensembles

déclaration de constantes

variables (état)

invariant (caractérisation de l'état)

assertions (théorèmes)

Partie dynamique

initialisation de l'état

opérations sur l'état

END



MACHINE ABSTRAITE : PARTIE STATIQUE

MACHINE *nom de la machine*

Partie en-tête

SETS

déclaration d'ensembles

CONSTANTS

déclaration de constantes

PROPERTIES

définition de propriétés sur les constantes

VARIABLES

déclaration de variables (état)

INVARIANT

définition de propriétés invariantes (caractérisation de l'état)

ASSERTIONS

Lemmes portant sur les variables

...

Partie dynamique

...

END



MACHINE ABSTRAITE : PARTIE STATIQUE

- Exemple



MACHINE ABSTRAITE : PARTIE DYNAMIQUE

- Utilisation de substitutions généralisées
- Structuration d'une opération

Syntaxe de la substitution	Nom de la substitution
BEGIN S END	bloc
PRE P THEN S END	pré-condition
ANY I WHERE G THEN S END	choix non borné

- Modification de variables

Notation	Nom
$skip$	substitution identité
$x := e$	substitution devient égal simple
$x, y := e, f$	substitution devient égal multiple
$x : (P)$	substitution devient tel que



MACHINE ABSTRAITE : FORME GLOBALE

```
MACHINE M (paramètre_1, ..., paramètre_n)  
CONSTRAINTS  
  X  
SETS  
  E  
CONSTANTS  
  c  
PROPERTIES  
  C  
VARIABLES  
  v  
INVARIANT  
  I  
ASSERTIONS  
  A de la forme  $A_1 \wedge A_2 \wedge \dots \wedge A_n$   
DEFINITIONS  
  D  
INITIALISATION  
  i  
OPERATIONS  
  u  $\leftarrow$  nom_op1 (w) = PRE P THEN S END ;  
  (...)   
END
```



MACHINE ABSTRAITE : EXEMPLE 1

```
MACHINE      Pixel

VARIABLES    x, y

INVARIANT
   $x \in 1..1280 \wedge y \in 1..1024$ 

INITIALISATION
   $x, y := 1, 1$ 

OPERATIONS

   $posX \leftarrow abs = posX := x;$ 

   $posY \leftarrow ord = posY := y;$ 

  random =
    ANY  $x', y'$ 
    WHERE  $x' \in 1..1280 \wedge y' \in 1..1024$ 
    THEN
       $x := x' \mid\mid y := y'$ 
    END;

  move (  $dx, dy$  ) =
    PRE  $dx \in \mathbb{Z} \wedge dy \in \mathbb{Z} \wedge (x + dx) \in 1..1280 \wedge (y + dy) \in 1..1024$ 
    THEN
       $x, y := (x + dx), (y + dy)$ 
    END

END
```



MACHINE ABSTRAITE : EXEMPLE 2

MACHINE
 Produit1
SETS
 PRODUIT
VARIABLES
 Produit, Quantité_en_stock
INVARIANT
 $Produit \subseteq PRODUIT \wedge$
 $Quantité_en_stock \in \text{Produit} \longrightarrow \mathbb{N}$
INITIALISATION
 $Produit := \emptyset \parallel$
 $Quantité_en_stock := \emptyset$



LE RAFFINEMENT

- Plusieurs raffinements peuvent être proposés pour une même spécification
- Dernier niveau = implantation
- Traduction automatique dans un langage de programmation



LE RAFFINEMENT

```
MACHINE  $M (par\_1, \dots, par\_n)$ 
CONSTRAINTS
   $X$ 
SETS
   $E$ 
CONSTANTS
   $c$ 
PROPERTIES
   $C$ 
VARIABLES
   $v$ 
INVARIANT
   $I$ 
ASSERTIONS
   $A$ 
DEFINITIONS
   $D$ 
INITIALISATION
   $i$ 
OPERATIONS
   $u \leftarrow op_1 (w) = \text{PRE } P \text{ THEN } S \text{ END};$ 
  (...)
END
```

```
REFINEMENT  $N (par\_1, \dots, par\_n)$ 
REFINES
   $M$ 
SETS
   $K$ 
CONSTANTS
   $d$ 
PROPERTIES
   $O$ 
VARIABLES
   $v'$ 
INVARIANT
   $J$ 
ASSERTIONS
   $A'$ 
INITIALISATION
   $i'$ 
OPERATIONS
   $u \leftarrow op_1 (w) = \text{PRE } Q \text{ THEN } T \text{ END};$ 
  (...)
END
```



UNE MACHINE ET SON RAFFINEMENT : LA MACHINE RESERVATION

```
MACHINE      RESERVATION
CONSTANTS    nb_max, SIEGES
PROPERTIES
    nb_max ∈ NAT1 ∧    SIEGES = 1..nb_max
VARIABLES    occupes, nb_libre
INVARIANT
    occupes ⊆ SIEGES ∧    nb_libre ∈ 0..nb_max ∧    nb_libre = nb_max - card(occupes)
INITIALISATION
    occupes, nb_libre := ∅, nb_max
OPERATIONS
    nb ← place_libre =
        BEGIN
            nb := nb_libre
        END ;
```



UNE MACHINE ET SON RAFFINEMENT : LA MACHINE RESERVATION

```
place ← reserver =  
  PRE nb_libre ≠ 0  
  THEN  
    ANY pp WHERE pp ∈ SIEGES — occupes THEN  
      place := pp || occupes := occupes ∪ {pp} || nb_libre := nb_libre — 1  
    END ;  
rr ← liberer(place) =  
  PRE place ∈ SIEGES  
  THEN  
    IF place ∈ occupes  
    THEN rr, occupes, nb_libre := TRUE, occupes — {place}, nb_libre + 1  
    ELSE rr := FALSE  
    END  
  END  
END
```

END



UNE MACHINE ET SON RAFFINEMENT : LA MACHINE RESERVATION1

```
REFINEMENT
  RESERVATION1
REFINES
  RESERVATION
VARIABLES
  etat, nb_libre
INVARIANT
   $etat \in SIEGES \rightarrow \text{BOOL} \wedge$ 
   $occupes = etat^{-1}[\{ \text{TRUE} \}]$ 
INITIALISATION
  nb_libre := nb_max ||
  etat := SIEGES * FALSE
```



UNE MACHINE ET SON RAFFINEMENT : LA MACHINE RESERVATION1

```
OPERATIONS
  place ← reserver =
    ANY pp1 WHERE pp1 ∈ etat-1[{FALSE}]
    THEN
      place := pp1 ||
      etat(pp1) := TRUE ||
      nb_libre := nb_libre - 1
    END ;
  rr ← liberer (place) =
    BEGIN
      IF etat(place) = TRUE
      THEN rr, nb_libre := TRUE, nb_libre + 1
      ELSE rr := FALSE
      END
      || etat(place) := FALSE
    END
END
END
```



FIN

